

Contesta cada pregunta de forma clara, breve y con tus propias palabras basándote en la experiencia de haber programado tu código.

Parte 1: El Formulario de Inicio (HTML y CSS)

Pregunta 1.1: El envío de datos sin JavaScript

En la página de inicio no se utilizó ninguna línea de código JavaScript para enviar la información.

- **Pregunta:** Explica cómo hace el navegador web, de forma completamente nativa, para saber a cuál dirección del servidor debe mandar el nombre y el apellido del usuario, y bajo cuál método de transferencia seguro lo hace. (Pista: Analiza los atributos de tu etiqueta `<form>`).

Pregunta 1.2: El espaciado con CSS

Durante la maquetación de los inputs de texto y el botón, configuraste propiedades de "Modelo de Caja" en tu archivo CSS.

- **Pregunta:** Explica cuál es la diferencia práctica en tu formulario entre haber usado `margin` (margen) y haber usado `padding` (relleno), y cómo ayudaron estas propiedades a que los elementos no se vieran pegados o desordenados.
-

Parte 2: El Servidor Local (Express y Rutas)

Pregunta 2.1: El papel del Middleware `urlencoded`

Para poder leer los datos que vienen del formulario, fue obligatorio activar la línea `app.use(express.urlencoded({ extended: true }))` en el archivo principal del servidor.

- **Pregunta:** Si borrras esa línea por accidente, ¿qué valor obtendrías al intentar imprimir un `console.log(req.body)` en tu ruta y por qué el servidor necesita este "ayudante" para poder leer los datos?

Pregunta 2.2: Separación de rutas con Express Router

En lugar de escribir la ruta de procesamiento dentro del archivo principal `app.js`, creaste un módulo independiente con `express.Router()`.

- **Pregunta:** ¿Por qué es una buena práctica de organización separar las rutas en archivos independientes en lugar de programar todas las funciones del servidor juntas en un solo archivo?
-

Parte 3: Comunicación con Internet (Fetch y Promesas)

Pregunta 3.1: El comportamiento de Async y Await

Tu servidor realiza dos peticiones externas a internet para traer una actividad aleatoria y la URL de la imagen de un animal.

- **Pregunta:** ¿Por qué la función de la ruta debe tener la palabra clave `async` y qué hace exactamente el servidor cuando se topa con la palabra clave `await` antes de cada `fetch`?

Pregunta 3.2: Extracción de datos específicos

Las APIs externas devuelven objetos JSON con mucha información que no necesitamos en la pantalla final.

- **Pregunta:** Explica brevemente cómo utilizaste la "notación de punto" en tu código (por ejemplo: `variable.propiedad`) para extraer únicamente el texto de la actividad y la dirección web de la imagen, ignorando las demás propiedades del JSON.
-

Parte 4: La Respuesta Dinámica

Pregunta 4.1: Crear HTML al vuelo desde el Backend

Al procesar la solicitud, el servidor no responde enviando un archivo JSON plano, sino que redacta y construye una estructura HTML completa desde cero usando `res.send()`.

- **Pregunta:** ¿Cuál es la diferencia para la experiencia del usuario final entre recibir un texto JSON crudo en la pantalla versus recibir una página HTML dinámica construida en el momento con su saludo y la tarjeta visual unificada?
-

Parte 5: Evidencias de Funcionamiento

Escribe abajo de cada enunciado el texto o enlace solicitado:

1. **Evidencia Visual:** Adjunta una captura de pantalla de la ventana del navegador web mostrando el resultado final obtenido: la tarjeta centrada con el saludo personalizado, la actividad destacada y la imagen controlada en su tamaño.